

기술 자료 보고서 (Technical Report)

IFAC 2026 자율주행 차량 경진대회

1. 팀 소개

본 팀은 F1TENTH 기반의 1/10 스케일 자율주행 레이싱 차량을 개발하였습니다. 본 문서는 차량의 하드웨어 구성 및 소프트웨어 스택에 대한 기술 개요를 제공합니다.

2. 하드웨어 구성 (Hardware Configuration)

2.1 플랫폼 구성 요소

구성 요소	사양
차량 플랫폼	Traxxas 4x4 (1/10 스케일)
구동 모터	Traxxas 3351R Velineon Brushless Motor
모터 컨트롤러	VESC MK VI (Vedder Electronic Speed Controller)
배터리	3S LiPo 배터리 1 개
라이다 센서	Hokuyo 10LX-H01 (2D LiDAR, 270° FOV, 40Hz, 최대 10m)

2.2 하드웨어 연결 구조

- 3S LiPo 배터리가 VESC MK VI 에 전원을 공급합니다.
- VESC MK VI 는 Traxxas Velineon 브러시리스 모터를 제어하며, USB/UART 를 통해 온보드 컴퓨터와 통신합니다.
- 온보드 컴퓨터(ROS 2 Humble)가 VESC 로 AckermannDriveStamped 명령을 전송하여 속도와 조향각을 제어합니다.
- Hokuyo 10LX-H01 LiDAR 는 USB/Ethernet 으로 온보드 컴퓨터에 연결되어 /scan 토픽으로 스캔 데이터를 제공합니다.

3. 소프트웨어 스택 (Software Stack)

3.1 운영 환경

항목	내용
운영체제	Ubuntu 22.04 LTS
미들웨어	ROS 2 Humble Hawksbill
주요 언어	C++ (런타임 노드), Python (런치 파일, 유틸리티)

3.2 소프트웨어 아키텍처 개요

자율주행 스택은 오프라인 전처리와 실시간 자율주행의 두 레이어로 구성됩니다. 오프라인에서 SLAM 맵을 기반으로 전역 레이스라인 웨이포인트를 생성하고, 실시간으로는 MCL 파티클 필터로 위치를 추정한 뒤 Pure Pursuit L1 컨트롤러가 조향·속도 명령을 생성합니다.

3.3 핵심 ROS 2 패키지 및 노드

3.3.1 Monte Carlo Localization (MCL) — `particle_filter_cpp`

역할: LiDAR 스캔과 오도메트리를 융합하여 맵 상의 차량 위치를 실시간으로 추정합니다.

알고리즘: Monte Carlo Localization (파티클 필터 기반)

주요 특징:

- 최대 1,000 개의 파티클로 확률적 위치 추정
- 이중 주파수 아키텍처: 고주파 오도메트리 업데이트 + 저주파 LiDAR 보정 (30 Hz)
- 운동 모델: 자전거 기구학(Bicycle Kinematics) + 속도 비례 잡음 모델
- 센서 모델: 4 성분 빔 모델 (Z_hit: 0.80, Z_short: 0.01, Z_max: 0.05, Z_rand: 0.14)
- 룩업 테이블 사전 계산으로 실시간 처리 최적화, OpenMP 병렬 레이캐스팅 지원

주요 토픽:

방향	토픽	메시지 타입
구독	/scan	sensor_msgs/LaserScan

구독	/odom	nav_msgs/Odometry
구독	/initialpose	geometry_msgs/PoseWithCovarianceStamped
발행	/pf/pose/odom	nav_msgs/Odometry
발행	/pf/viz/particles	geometry_msgs/PoseArray

주요 파라미터 (mcl_config.yaml):

- max_particles: 1000
- angle_step: 18 (LiDAR 다운샘플링 비율)
- timer_frequency: 30.0 Hz
- wheelbase: 0.324 m (측거)
- sigma_hit: 7.0 (센서 측정 노이즈 표준편차)

3.3.2 웨이포인트 퍼블리셔 — wpnt_publisher

역할: 전역 레이스라인에서 현재 위치 기준 앞쪽 N 개의 로컬 웨이포인트를 추출하여 컨트롤러에 제공합니다.

주요 특징:

- 전역 웨이포인트와 Frenet 오도메트리 기반 위치에서 로컬 구간 추출
- 장애물 회피 경로(OT, Obstacle Trajectory) 우선 출력 지원
- 기본 웨이포인트 수: 50 개

주요 토픽:

방향	토픽	메시지 타입
구독	/global_waypoints	f110_msgs/WpntArray
구독	/car_state/frenet/odom	nav_msgs/Odometry
발행	/local_waypoints	f110_msgs/WpntArray
발행	/local_waypoints/path	nav_msgs/Path

3.3.3 MAP 컨트롤러 — new_map_con

역할: 로컬 웨이포인트를 추종하여 조향각과 목표 속도를 생성합니다.

알고리즘: Pure Pursuit L1 경로 추종

L1 거리 계산: $L1_distance = q_l1 + speed_now \times m_l1$, 횡방향 오차와 속도에 따라 클리핑 적용

조향각 계산: $lateral_accel = 2 \cdot v^2 / L1 \cdot \sin(\eta) \rightarrow$ Pacejka 타이어 모델 기반 룩업 테이블로 조향각 결정

주요 특징:

- 속도 적응형 L1 거리 (고속 시 전방 주시점 자동 확장)
- 횡방향 오차 기반 L1 하한 클리핑
- 조향각 변화율 제한 (0.4 rad/iteration)
- Pacejka 타이어 모델 기반 조향 룩업 테이블 적용

주요 토픽:

방향	토픽	메시지 타입
구독	/local_waypoints	f110_msgs/WpntArray
구독	/pf/pose/odom	nav_msgs/Odometry
구독	/car_state/frenet/odom	nav_msgs/Odometry
발행	/drive	ackermann_msgs/AckermannDriveStamped

주요 파라미터:

- $l1_distance$: 0.3 m (기본 L1 거리)
- $l1_gain$: 0.5 (속도 비례 L1 계인)
- t_clip_min : 1.5 m (L1 최솟값)
- t_clip_max : 5.0 m (L1 최댓값)

3.3.4 오프라인 레이스라인 생성기 – *offline_trajectory_generator*

역할: SLAM 으로 생성한 점유 격자 맵에서 최적 레이스라인 웨이포인트를 오프라인으로 생성합니다.

처리 파이프라인:

- 점유 격자 맵(OccupancyGrid) 로드
- 중앙선 스켈레톤 추출 (거리 변환 기반)
- 좌우 경계선 생성 (충돌 검사 포함)
- 균일 간격 웨이포인트 리샘플링 (기본: 0.1 m)
- 곡률 기반 속도 프로파일 생성: $v_{\max_curve} = \sqrt{a_{\text{lat_max}} / |\kappa|}$
- JSON / CSV 형식으로 출력

주요 파라미터 (gui_params.yaml):

- waypoint_step: 0.1 m (웨이포인트 간격)
- safety_width: 0.35 m (차량 폭 + 안전 마진)
- max_speed: 4.0 m/s
- min_speed: 1.0 m/s
- max_lateral_accel: 8.0 m/s²

3.3.5 랩 타이머 — lap_timer

역할: Frenet s 좌표 기반 실시간 랩 타이밍

- Frenet 좌표계의 s 값 래핑(wraparound) 감지로 완주 판정
- 현재 랩 타임 및 최고 랩 타임 퍼블리시 (std_msgs/Float64)
- RViz2 HUD 오버레이 디스플레이

3.4 커스텀 메시지 타입 — f110_msgs

메시지	주요 필드
Wpnt	s_m, d_m, x_m, y_m, psi_rad, kappa_radpm, vx_mps, ax_mps2
WpntArray	Wpnt[] 배열
OTWpntArray	장애물 회피 경로 웨이포인트 배열
Obstacle	s_start, s_end, d_right, d_left

3.5 Frenet 좌표 변환 — frenet_converter

경로 추종 및 위치 표현에 Frenet 좌표계를 사용합니다. s 는 레이스라인을 따르는 종방향 거리(m), d 는 레이스라인으로부터의 횡방향 편차(m)입니다. 3 차 스플라인 보간 기반 정밀 변환을 제공하며, 속도 벡터도 Frenet 프레임으로 변환합니다 ($s_{\dot{}}$, $d_{\dot{}}$).

4. 전체 시스템 데이터 플로우

단계	모듈	출력 토픽
1. 센서 입력	Hokuyo 10LX-H01 LiDAR	/scan
2. 위치 추정	MCL 파티클 필터	/pf/pose/odom
3. 웨이포인트 추출	wpnt_publisher	/local_waypoints
4. 제어 명령 생성	MAP 컨트롤러 (Pure Pursuit L1)	/drive
5. 모터 구동	VESC MK VI	(하드웨어 출력)

5. 빌드 및 실행 방법

빌드:

- colcon build --symlink-install

위치 추정 실행 (실차 모드):

- ros2 launch particle_filter_cpp mcl_launch.py mod:=real map_name:=<맵이름>

MAP 컨트롤러 실행:

- ros2 launch new_map_con map_controller.launch.py

랩 타이머 실행:

- ros2 launch lap_timer lap_timer.launch.py

본 문서는 parkasd/2026_IFAC 저장소의 현재 브랜치 기준으로 작성되었습니다.